

Checkpoint 1 — Especificação Léxica e Analisador Léxico

1. Contexto e objetivo

Este é o primeiro entregável do projeto do semestre: o compilador da linguagem que estão definindo, dentro do escopo definido (variáveis de tipos básicos, estruturas de controle e funções). Todo o resto do projeto — parser, análise semântica, geração de código para a TAM — depende do que for construído nesta primeira entrega. Um scanner mal especificado ou copiado sem entendimento vira dívida técnica em todas as entregas seguintes.

Por isso a entrega tem duas partes que se cobram mutuamente: primeiro vocês especificam, com precisão, o que é um token válido na própria linguagem (Parte 1); depois implementam um analisador léxico que reconhece exatamente essa especificação, construído a partir do autômato finito determinístico (AFD) correspondente a cada categoria de token (Parte 2). O objetivo não é "ter um scanner que funciona" — é ter um scanner cujo código o grupo consegue justificar, estado por estado, em uma eventual arguição oral.

2. Parte 1 — Especificação léxica da linguagem

Entreguem um documento curto (Markdown ou PDF, 1–3 páginas) descrevendo formalmente o léxico da sua linguagem. Ele deve cobrir, no mínimo, as cinco categorias de token trabalhadas em aula:

Token	Notação (regex/EBNF)	Exemplos válidos	O que precisa decidir
Identificador	letra (letra dígito "_")*	total, x1, contaTens	Case-sensitive? "_" permitido? tamanho máximo?
Palavra reservada	mesmo padrão do identificador + tabela de busca	if, while, int, return	Lista fechada de palavras-chave da sua linguagem
String	" (qualquer caractere ≠ ") * "	"ok", "linha 1"	Escapes (\)? o que fazer se não fechar até EOF/fim de linha?
Operador	1 ou 2 caracteres (ex.: =, ==, <, <=, !=)	=, ==, <=, +, &&	Quais operadores existem? quais têm forma composta?
Literal numérico	dígito+ ("." dígito+)?	10, 3.14, 0	Notação científica? hexadecimal? são bônus, não obrigatórios

Além da tabela acima, preenchida com a notação real da sua linguagem, o documento precisa responder explicitamente:

- Qual é o alfabeto de entrada (que caracteres são válidos em algum ponto de algum token)?
- A linguagem é case-sensitive? Palavras reservadas e identificadores seguem a mesma regra de maiúsculas/minúsculas?
- Como são delimitados espaços em branco e comentários (linha e/ou bloco)? Comentários de bloco podem ser aninhados?
- Para operadores com forma composta (ex.: = vs. ==), qual é a regra de desambiguação — "maximal munch": o scanner sempre tenta consumir o prefixo válido mais longo antes de decidir o token.
- Quais são exatamente as palavras reservadas da linguagem (lista fechada, não apenas exemplos)?

Esta especificação é o "contrato" do seu scanner: o código da Parte 2 deve implementar exatamente o que este documento descreve, nem mais nem menos. Ela também é o ponto de partida para a gramática (BNF/EBNF) que vocês vão escrever para a análise sintática — inconsistências aqui custam retrabalho lá na frente.

3. Parte 2 — Analisador léxico como autômato finito

3.1 O que precisa ser implementado

O scanner desta entrega deve ser escrito à mão em Java, diretamente a partir do AFD de cada categoria de token — sem usar um gerador JFlex.

Estrutura mínima esperada (os nomes são sugestões, adapte à sua linguagem):

- **class Token** — tipo do token, lexema (texto original), linha e coluna onde começa.
- **class Scanner** — mantém um cursor sobre o código-fonte, com `peek()` (olha o caractere atual sem consumir), `advance()` (consome e move o cursor) e `hasNext()`.
- **nextToken()** — devolve o próximo token, aplicando "maximal munch": consome o maior prefixo do texto restante que ainda forma um token válido.
- Cada estado do AFD de cada categoria de token deve corresponder a um trecho identificável do código (um case de switch, ou um estado no padrão State) — não a uma função só que "faz tudo" por regex de biblioteca.

3.2 Tratamento de erros

Um caractere inválido ou uma string não fechada são erros léxicos, não motivo para o programa travar (exception não tratada) ou para o scanner parar de funcionar. O requisito mínimo:

- Reportar o erro com linha e coluna onde ele ocorre.
- Seguir para o próximo token após reportar o erro (recuperação), continuando a tokenizar o restante do arquivo.
- Tratar corretamente o fim de arquivo (EOF) no meio de uma string ou de um comentário de bloco — isso também é erro léxico, não deve ser ignorado silenciosamente.

3.3 Documentação do autômato

Entreguem, junto com o código, o desenho do AFD (pode ser à mão, fotografado, ou feito em qualquer ferramenta). Cada estado do desenho deve ter o mesmo nome (ou uma correspondência óbvia) do estado equivalente no código — é isso que permite a vocês apontar "esta transição no desenho é esta linha no código".

3.4 Suíte de testes

Escrevam um conjunto de testes automatizados (JUnit ou uma classe main com asserts simples) cobrindo:

- Pelo menos um caso válido de cada uma das cinco categorias de token.
- Pelo menos três casos de erro: string não fechada até EOF, string não fechada até fim de linha (se a linguagem não permitir strings multilinha), e um caractere fora do alfabeto da linguagem.
- Um trecho de código "realista" (uma linha com várias declarações/expressões) tokenizado corretamente do início ao fim, incluindo espaços e comentários misturados.

4. Entregáveis — checklist

- [] Documento de especificação léxica em Markdown ou PDF.
- [] Código-fonte do scanner (Java), compilável e executável.
- [] Desenho do AFD para identificador/palavra-reservada e para string (Seção 3.3).
- [] Suíte de testes com casos válidos e de erro (Seção 3.4), com evidência de execução (print/log da saída).
- [] Log de uso de IA (Seção 7), mesmo que seja "não usamos IA nesta etapa".

5. Formato de entrega

Organizem os arquivos num único repositório (ou pasta compactada) com esta estrutura mínima:

```
cp1-<nome-do-grupo>/
  especificacao-lexica.md
  log-uso-ia.md
  automatos/      (desenhos do AFD)
  src/             (código Java do scanner)
  test/           (suíte de testes)
```

Entregas incompletas na data podem ser reapresentadas na próxima entrega, com desconto.

6. Lembrete — escopo fixo da linguagem

A especificação léxica precisa, no mínimo, dar conta de tokenizar programas que usem (mesmo que a sintaxe concreta de cada construção ainda não esteja definida pelo grupo):

- Tipos básicos: int, double, bool, char, string.
- Estruturas de controle: seleção (if/else) e repetição (while e/ou equivalente).
- Procedimentos e funções: declaração, parâmetros, chamada e retorno (com e sem valor).

7. Uso de IA

Uso de IA é permitido e esperado, mas registrado — não presumido. A partir desta entrega, o log de uso de IA é obrigatório.

Permitido	Vedado sem declarar
Tirar dúvida de sintaxe Java; gerar boilerplate de leitura de arquivo/stream	Pedir para a IA gerar o scanner inteiro a partir da especificação de tokens do grupo
Pedir explicação de um erro de compilação específico	Colar a especificação léxica inteira e pedir "implemente o AFD"
Revisar/explicar um trecho do próprio código já escrito	Entregar código que o grupo não consegue explicar na defesa oral

8. Critérios de avaliação

Critério	O que é observado
Especificação léxica	Documento cobre as 5 categorias de token, coerente com o escopo fixo da linguagem (tipos, controle, funções), com notação regex/EBNF e tabela de palavras reservadas
Correção do scanner	Reconhece corretamente identificadores, palavras reservadas, strings, operadores e literais numéricos na linguagem do grupo
Tratamento de erros	Erros léxicos (string não fechada, caractere inválido) são reportados com linha/coluna e o scanner segue para o próximo token, sem travar
Testes	Suíte com casos válidos e casos de erro, executável, com evidência de execução (saída ou log)